

LRS™ — Point d'étape et feuille de route

Sortie de Streamlit vers une architecture Docker + Vercel — 7 septembre 2026

1. D'où on part

Le déclencheur initial : le déploiement de l'app Streamlit (**app.py**) sur Vercel échouait ("n'exporte pas de variable app/application/handler") — incompatibilité architecturale de fond, Streamlit étant un serveur persistant et Vercel une plateforme de fonctions serverless sans état, pas un bug ponctuel à corriger.

Première étape réalisée : migration du moteur d'audit d'OpenAI vers Anthropic (Claude), dans **app.py** puis dans le pilote FastAPI (**audit_engine.py**, **pilot_server.py**).

En creusant la question de la refonte visuelle (le vrai motif du départ de Streamlit : liberté totale sur le design, ce que Streamlit ne permet pas avec ses widgets natifs), une découverte majeure a changé la donne : une **branche distante** (44 commits d'avance sur main) contenait déjà un travail bien plus abouti que prévu.

2. Ce qui existait déjà (branche distante, découverte le 7 septembre)

- Un pilote FastAPI + frontend custom ("Apple-style") couvrant déjà les **8 onglets** de l'app Streamlit (Dashboard, Audit, Multi-Audit, Suivi, Historique, Ressources, Creative Studio, Plans) — pas seulement l'audit.
- Un **vrai système d'abonnement Stripe** : paiement → webhook → compte activé → lien magique de connexion envoyé par email (**user_accounts.py**, **email_alerts.py**), en remplacement du mot de passe bêta partagé.
- Une **page de vente (sales-site/**, Next.js) avec le copy marketing final, bilingue FR/EN, bandeau de consentement cookies RGPD, pixels Meta/Google prêts (désactivés tant qu'aucun ID n'est fourni).
- Un durcissement sécurité déjà en place : protection SSRF sur les URLs auditées, rate-limiting du login, déduplication anti-rejeu des webhooks Stripe.
- Un squelette de déploiement **Docker** (non testé) pour les 3 services (pilote, Streamlit, service de webhook Stripe).

Décision prise : **adopter cet état comme référence** plutôt que de reconstruire (Vercel + Supabase) ce qui existait déjà en plus mature. Les commits locaux de cette session portant sur la voie Vercel ont été abandonnés (rien n'était perdu, jamais poussé).

3. Ce qui a été testé et corrigé aujourd'hui (conditions réelles)

Le document de passation de la session précédente notait explicitement que **rien n'avait pu être testé avec de vrais identifiants** Stripe/SMTP (sandbox sans accès réseau). C'est ce qui a été fait aujourd'hui, avec de vrais identifiants de test.

Parcours validé de bout en bout :

- Produit + prix créés dans le sandbox Stripe (50€/mois)
- Paiement réel avec carte de test Stripe (4242 4242 4242 4242)
- Webhook signé reçu et traité → compte activé en base

- Email envoyé via un vrai SMTP (Gmail, mot de passe d'application)
- Lien magique cliqué → app Streamlit déverrouillée

3 bugs bloquants découverts et corrigés :

- **stripe.api_key** n'était jamais configuré → l'endpoint **/checkout/beta** plantait systématiquement (AuthenticationError).
- Le webhook appelait **.get()** sur l'objet SDK Stripe renvoyé par **construct_event()** (pas un dict) → **tous** les événements webhook réels échouaient en erreur 500.
- **LRS_USERS_DB_PATH** vide dans **.env** → connexion à une base SQLite temporaire différente à chaque appel ("no such table"), à cause d'un **os.environ.get(var, default)** qui ne retombe sur le défaut que si la variable est absente, pas si elle est vide.

Ajouté par la même occasion : un reverse proxy **Caddy** (TLS automatique Let's Encrypt) devant les 3 services Docker — prérequis pour que Stripe puisse atteindre le webhook en production.

4. Architecture cible

Surface	Techno	Hébergement
Page de vente (marketing)	Next.js	Vercel
App principale + pilote + webhook Stripe	Streamlit + FastAPI	VPS (Docker + Caddy)

Les deux surfaces sont indépendantes : la page de vente pointe vers un **Payment Link Stripe** direct (pas vers l'API du VPS), donc déployable et modifiable sans dépendre du serveur backend.

Écart connu : le déblocage par abonnement Stripe (lien magique) n'existe aujourd'hui que dans **app.py/Streamlit**. Le pilote FastAPI a son propre système de mot de passe partagé, mais pas encore cette logique — à porter si le pilote doit devenir l'interface principale.

5. Feuille de route — ce qu'il reste à faire

Tâche	Bloqué sur	Statut
Ré-autoriser le CLI Stripe pour le Live	Action utilisateur (lien envoyé)	En attente
Créer produit/prix Live + récupérer sk_live_...	Ré-autorisation ci-dessus	En attente
Acheter un nom de domaine	Décision/achat utilisateur	Mis de côté
Créer une VM Google Cloud (Always Free, e2-micro)	Décision/compte utilisateur	Mis de côté
Déployer Docker (pilote + Streamlit + webhook + Caddy) sur la VM créée	VM créée	Prêt côté code
Configurer le vrai webhook Stripe (Dashboard, mode Live)	Domaine + VM + DNS	Bloqué
Déployer sales-site/ sur Vercel	Rien (indépendant)	Prêt à lancer
Renseigner NEXT_PUBLIC_STRIPE_LINK avec le Payment Link Live	Rien	Fait en local
Faire relire le contenu légal (privacy/terms) par un professionnel	Action utilisateur	En attente

Porter le lien magique Stripe dans le pilote FastAPI	Décision produit	Non planifié
--	------------------	--------------

6. Points d'attention

- Ne pas publier le lien de la page de vente avant que le webhook (VPS) soit joignable — un paiement reçu sans webhook actif n'active pas automatiquement le compte (récupérable manuellement depuis le Dashboard Stripe, mais pas automatique).
- Le mot de passe bêta local ("change-moi", dans `.streamlit/secrets.toml`) est un placeholder de dev — à changer avant toute exposition publique de l'app Streamlit.
- Toute la config Live (clé secrète, `price_id`, webhook secret) est distincte de la config Test déjà en place — ne pas mélanger les deux dans le même `.env` de production.